SENTINEL: Technical Requirements & Environment Guide

---

1. LANGUAGES & FRAMEWORKS

Overview Table

Component	Language	Framework/Tools	Why (Big Tech Usage)
Entropy Agent	**Go 1.22+**	client-go, cobra	Google, Uber, Docker, Kubernetes itself - all written in Go
API Server	**Go 1.22+**	Fiber or Gin	Used by Google, Cloudflare, Twitch for high-performance APIs
Purge Controller	**Go 1.22+**	controller-runtime, client-go	All K8s operators at Google, Red Hat, VMware use this
Frontend	**TypeScript 5+**	React 18, Vite	Meta, Airbnb, Netflix, Uber - industry standard
Styling	**CSS**	Tailwind CSS 3	Shopify, Netflix, OpenAI - rapid UI development
UI Components	**TypeScript**	shadcn/ui (Radix)	Built on same primitives as Vercel, Linear, Cal.com
Charts	**TypeScript**	Recharts or D3.js	Airbnb (Visx), Netflix, Uber use D3-based charting
State Management	**TypeScript**	Zustand + React Query	Adopted by many teams moving away from Redux
Database	**SQL**	SQLite → PostgreSQL	SQLite for local; Postgres in prod (everyone uses it)
Caching/Pubsub	**-**	Redis	Twitter, GitHub, Instagram, Snapchat
Infrastructure	**YAML + HCL**	Kubernetes, Helm, Terraform	Google, Amazon, Microsoft - all K8s native
CI/CD	**YAML**	GitHub Actions	Microsoft/GitHub, free for open source
Containerization	**Dockerfile**	Docker, containerd	Universal standard

---

2. WHY THESE CHOICES? (Big Tech Alignment)

Go for Backend - The Kubernetes Native Language

```
┌─────────────────────────────────────────────────────────────┐
│                    GO IN PRODUCTION                         │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  Google:     Kubernetes, gRPC, Istio, Vitess                │
│  Uber:       Core infrastructure, geofencing                │
│  Cloudflare: Edge computing, DNS                            │
│  Docker:     Docker engine, containerd                      │
│  Dropbox:    Performance-critical services                  │
│  Twitch:     Video infrastructure                           │
│  Netflix:    Container orchestration (Titus components)     │
│  Meta:       Various infrastructure tools                   │
│                                                             │
│  WHY GO:                                                    │
│  ✓ Compiles to single binary (easy container deployment)    │
│  ✓ Built-in concurrency (goroutines)                        │
│  ✓ Low memory footprint                                     │
│  ✓ Native Kubernetes client libraries                       │
│  ✓ Fast compilation                                         │
│  ✓ Strong typing without complexity                         │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

TypeScript + React for Frontend

```
┌─────────────────────────────────────────────────────────────┐
│              TYPESCRIPT + REACT IN PRODUCTION               │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  Meta:       Created React, uses it everywhere              │
│  Airbnb:     React + TypeScript, major contributors         │
│  Netflix:    React for TV UI, web platform                  │
│  Uber:       React + TypeScript across all web apps         │
│  Microsoft:  TypeScript creator, VS Code, Azure Portal      │
│  Slack:      Full TypeScript migration completed            │
│  Shopify:    React + TypeScript for admin and storefront    │
│  Discord:    React + TypeScript                             │
│                                                             │
```

```
WHY REACT + TYPESCRIPT:
✓ Type safety catches bugs before runtime
✓ Huge ecosystem and community
✓ Component-based architecture
✓ Excellent developer tooling
✓ Easy to hire for / learn
```

---

3. OPERATING SYSTEM REQUIREMENTS

Recommended: Linux (Native or WSL2)

OS Option	Recommended?	Notes
Ubuntu 22.04/24.04 (Native)	✅ Best	Cleanest experience, everything works out of box
WSL2 + Ubuntu	✅ Great	Windows users - this is the way
Fedora / Arch	✅ Good	Works fine, slightly different package managers
macOS (Intel/M1/M2)	✅ Good	Works well, use Homebrew
Windows (Native)	⚠️ Avoid	Docker/K8s have issues, use WSL2 instead

Why Linux?

```
Kubernetes, Docker, and most cloud-native tools are built
for Linux first. Running on Linux means:

✓ No virtualization overhead (native containers)
✓ Same environment as production
✓ All tools work without workarounds
✓ Better resource efficiency
✓ Industry standard for backend development
```

---

4. HARDWARE REQUIREMENTS

Running Directly on Laptop

Resource	Minimum	Recommended	Notes
CPU	4 cores	8+ cores	More cores = more pods
RAM	8 GB	16 GB	K8s + containers are memory hungry
Disk	30 GB free	50 GB+ free	Container images add up
Architecture	x86_64 / ARM64	x86_64 preferred	M1/M2 Macs work fine

Running in a VM (if you prefer isolation)

Resource	Allocation	Host Should Have
vCPUs	4 vCPUs	6+ physical cores
RAM	8 GB	12+ GB total
Disk	40 GB	60+ GB free
Network	Bridged/NAT	Either works
Hypervisor	VirtualBox/VMware/KVM	VirtualBox is free

VM Setup (If Using)

```bash
```

```
# Recommended VM Specs for VirtualBox/VMware:
- OS: Ubuntu 22.04 LTS Server or Desktop
- CPUs: 4
- RAM: 8192 MB
- Disk: 40 GB (dynamically allocated)
- Network: NAT with port forwarding OR Bridged
- Graphics: Minimal (server) or 128MB (desktop)

# Port forwards needed (if NAT):
- Host 3000  ⟶ Guest 3000  (UI)
- Host 8080  ⟶ Guest 8080  (API)
- Host 6443  ⟶ Guest 6443  (K8s API - optional)
```

---

5. COMPLETE SOFTWARE STACK (All Free & Open Source)

Core Development Tools

Tool	Version	Purpose	License	Cost
Go	1.22+	Backend language	BSD-3	Free
Node.js	20 LTS	Frontend tooling	MIT	Free
npm/pnpm	Latest	Package manager	MIT	Free
Git	2.40+	Version control	GPL-2	Free
Docker	24+	Container runtime	Apache 2.0	Free
kubectl	1.29+	K8s CLI	Apache 2.0	Free
minikube	1.32+	Local K8s cluster	Apache 2.0	Free
Helm	3.14+	K8s package manager	Apache 2.0	Free

Development Environment

Tool	Purpose	License	Cost
VS Code	Code editor	MIT	Free

Tool	Purpose	License	Cost
Go extension	Go support in VS Code	MIT	Free
ESLint/Prettier	Code quality	MIT	Free
k9s	Terminal K8s UI	Apache 2.0	Free
Lens	Desktop K8s IDE	MIT (Open Lens)	Free

Runtime Dependencies

Tool	Purpose	License	Cost
SQLite	Embedded database	Public Domain	Free
Redis	Caching/pubsub	BSD-3	Free
PostgreSQL	Production database	PostgreSQL License	Free
Nginx	Sample app / ingress	BSD-2	Free

Frameworks & Libraries

Library	Purpose	License	Cost
client-go	K8s Go client	Apache 2.0	Free
Fiber	Go web framework	MIT	Free
React	UI framework	MIT	Free
Tailwind CSS	Styling	MIT	Free
shadcn/ui	UI components	MIT	Free
Recharts	Charts	MIT	Free
Zustand	State management	MIT	Free
React Query	Server state	MIT	Free

6. WHAT BIG TECH USES (Direct Comparison)

---

7. TECHNIQUES & PATTERNS (Enterprise Grade)

Architecture Patterns We'll Use

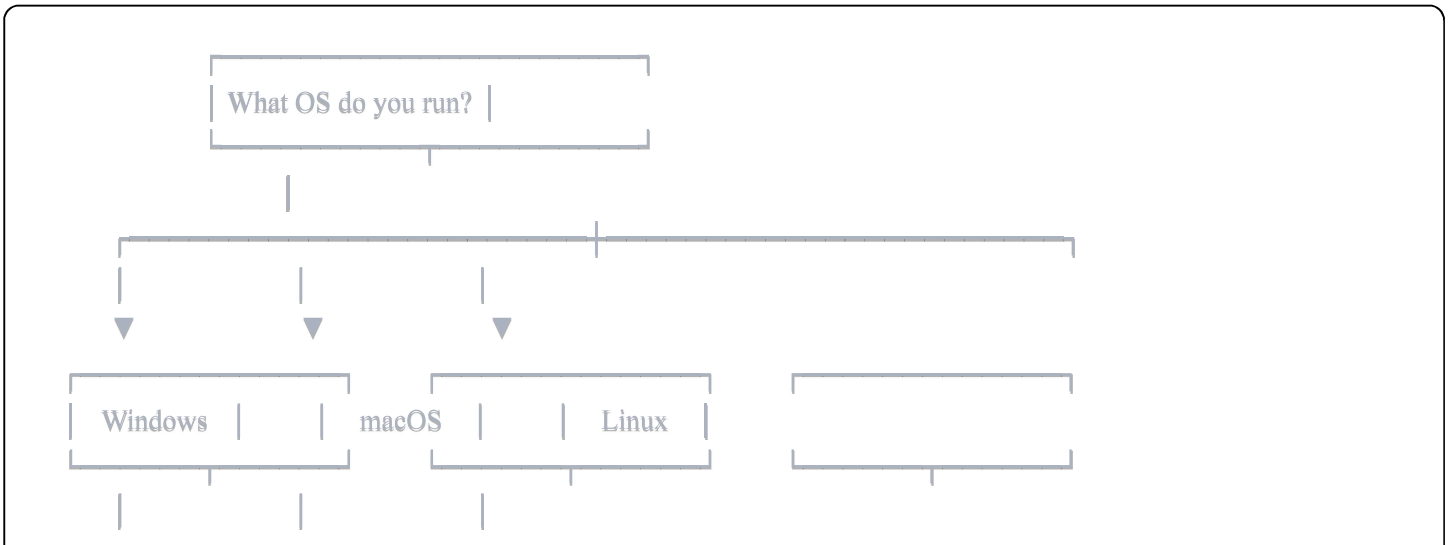
Pattern	What It Is	Where Used
DaemonSet	Agent runs on every node	Datadog, Prometheus, Falco
Operator Pattern	Custom controller for K8s	All major K8s tools

Pattern	What It Is	Where Used
Sidecar Pattern	Helper container alongside main	Istio, Envoy
Event-Driven	React to changes, not poll	Modern microservices
GitOps	Git as source of truth	ArgoCD, Flux, Weaveworks
12-Factor App	Cloud-native principles	Heroku, Netflix, everyone

Coding Practices

Practice	Description
Clean Architecture	Separation of concerns, testable code
Dependency Injection	Loosely coupled components
Interface-Driven Design	Program to interfaces, not implementations
Structured Logging	JSON logs, correlation IDs
Graceful Shutdown	Handle SIGTERM properly
Health Checks	Liveness + Readiness probes
Metrics Exposition	Prometheus-compatible /metrics

8. ENVIRONMENT SETUP DECISION TREE



---

9. SUMMARY

Final Stack Confirmation

Layer	Choice
OS	Linux (Ubuntu 22.04) - native or WSL2
Backend	Go 1.22+

Layer	Choice
Frontend	TypeScript + React 18 + Vite
Database	SQLite (dev) → PostgreSQL (if needed)
Kubernetes	minikube (local)
All other tools	100% open source, $0 cost

Prerequisites Checklist

- [ ] 8GB+ RAM available
- [ ] 30GB+ disk space free
- [ ] Linux/macOS/WSL2 environment
- [ ] Internet connection for package downloads

---

10. READY TO INSTALL?

Once you confirm:

1. **Your OS choice** (native Linux, WSL2, or macOS)
2. **Your hardware specs** (RAM, CPU, disk)
3. **Any preferences** on tools

I'll generate the exact installation script for your environment and we'll start building!